[4]

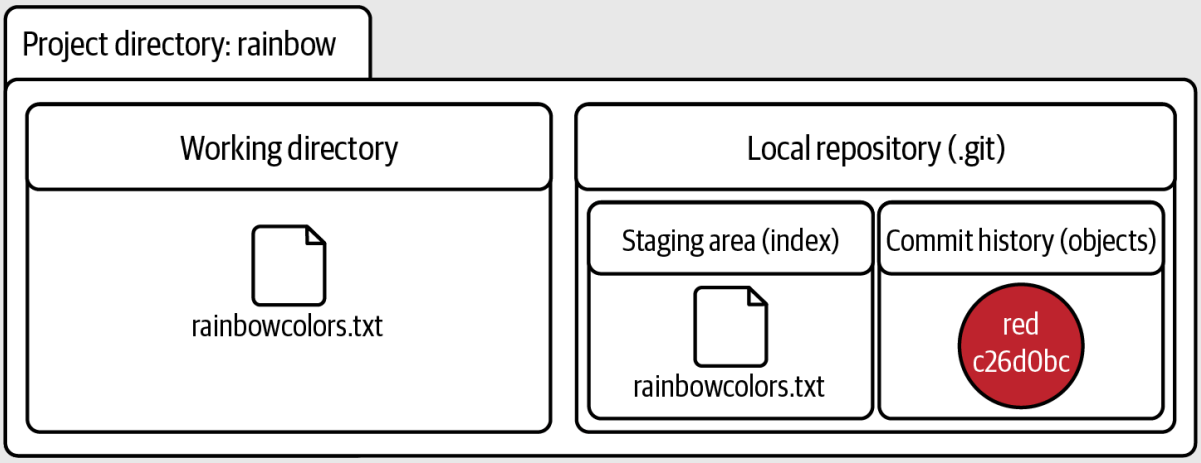
Branches

In the last chapter, you learned about the process of making a commit and you made your first commit in the rainbow repository.
In this chapter, you'll learn what branches are and why we use them. You will continue making commits in the rainbow repository and see how this affects the branches in your project. Finally, you will make a new branch and learn how to switch (or change) onto it. In addition, in the process of making more commits in the rainbow repository you will learn about the concept of unmodified and modified files and how commits are linked to one another.

State of the Local Repository

In Chapter 2, we built a Git Diagram with the four important areas of Git: the working directory, the staging area, the commit history, and the local repository. Visualize it 4-1 uses the Git Diagram to show the state of the rainbow repository at the start of this chapter.

[VISUALIZE IT 4-1]



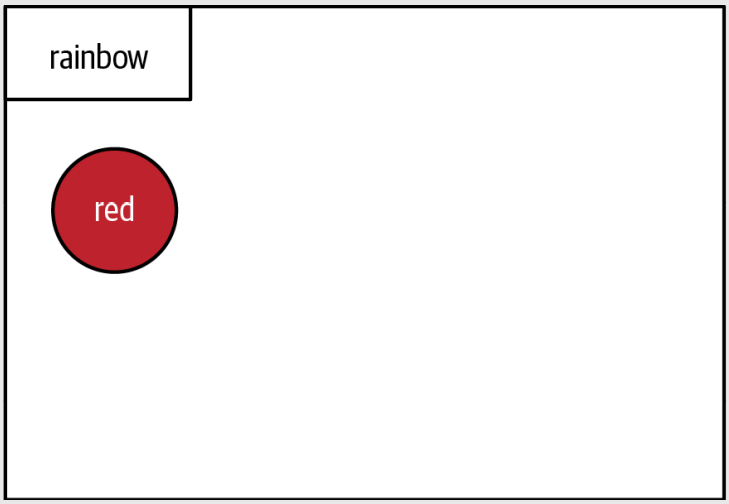
The Git Diagram showing the state of the rainbow repository at the start of Chapter 4, with one commit, the red commit

[NOTE]

From this point forward, the Visualize It diagrams will only display the name of the color in the commit, or an abbreviation of the name. I will no longer include the first seven characters of the commit hash.

To focus on the commit history, I'm now going to introduce a new diagram called the Repository Diagram. The Repository Diagram includes only a representation of the commit history of a repository and the relevant branches and references. The local repository is represented by a rectangle, with the name of the repository in the top-left corner. Visualize it 4-2 shows the current state of the rainbow repository in the form of a Repository Diagram.

[VISUALIZE IT 4-2]



The Repository Diagram showing the current state of the rainbow repository with one commit, the red commit

Why Do We Use Branches?

Before we get into the specifics of branches in Git, I want to explain why they're so useful. There are two main reasons to use branches:

- To work on the same project in different ways.
- To help multiple people work on the same project at the same time.

You can think of a branch like a line of development. A Git project can have multiple branches (or lines of development). Each of these branches is a standalone version of the project. Different Git projects can use branches in different ways, depending on the needs of the people working on the project.

One common pattern for working with branches is to have one official primary line of development—the main or primary branch—and off of that to create secondary branches, called *topic branches* or *feature branches*, that are used to work on just a specific part of the project. These topic branches are short-lived; they are ultimately combined or incorporated back into the primary branch and then deleted. The two processes you can use to integrate one branch into another are called merging and rebasing. We will cover these in more depth in Chapters 5, 9, 10, 11, and 12 (yup, they are big topics!).

This pattern of having one primary line of development is the approach you will take in the Rainbow project. To make this a little more concrete, let's take a look at Example Book Project 4-1 to see how branching might be used in the Book project.

EXAMPLE BOOK PROJECT 4-1

Suppose the official branch of my Book project is the main branch. I don't want to add work to the official line of development until my editor has reviewed and approved it. So, each time I work on a chapter, I can make a secondary branch, work on that branch, and then submit it to be reviewed by my editor. Once they've approved the work that I have done in that secondary branch, I can combine that branch into the main branch.

If at some point I decide to work on the book with a coauthor, we can each work on our own secondary branches. Then, only when the work on a given secondary branch has been approved by both the other author and the editor will it be considered ready to be combined with the main branch.

Now that you have an idea of why we use branches, let's get a clearer sense of how they work in Git.

WHAT EXACTLY ARE BRANCHES IN GIT?

Branches in Git are movable pointers to commits. When you list the commits in a local repository using the git log command, you can see information about which branches point to which commits. Go to Follow Along 4-1 to see this explicitly in the rainbow repository.

[FOLLOW ALONG 4-1]

```
1 rainbow $ git log
  commit c26d0bc371c3634ab49543686b3c8f10e9da63c5 (HEAD -> main)
  Author: annaskoulikari <gitlearningjourney@gmail.com>
  Date: Sat Feb 19 09:23:18 2022 +0100
  red
```

What to notice:

- In the git log output, next to the commit hash inside the parentheses you see HEAD -> main.

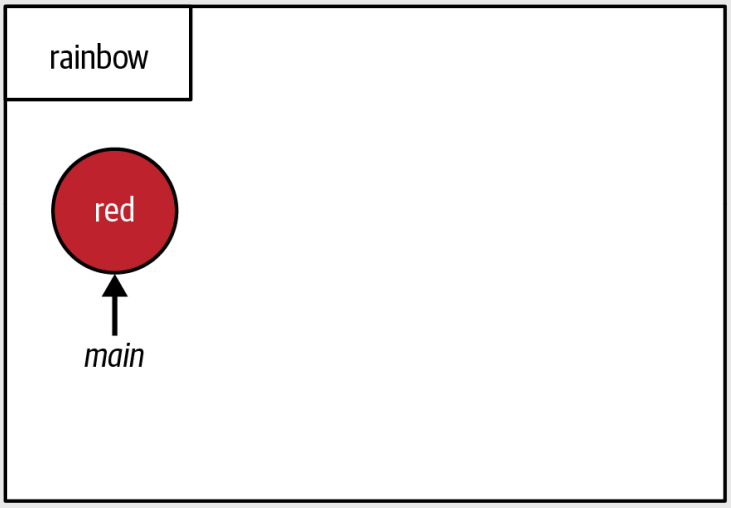
The branch or branches that appear inside the parentheses next to a particular commit hash in the git log output are the branches that point to that commit.

[NOTE]

HEAD (in capital letters) is not a branch. We'll discuss what it is in "What Is HEAD?" on page 55.

In the rainbow repository, the main branch points to the red commit. This is illustrated in Visualize it 4-3.

[VISUALIZE IT 4-3]



The main branch points to the red commit in the rainbow repository

Later in this chapter, you will continue making commits on the main branch, and you will see how the main branch will move to point to the new commits.

[NOTE]

The main branch in the rainbow repository is a local branch. In the first part of this book, when you are working only with a local repository in the Rainbow project, you will work only with local branches. In the second part of the book, once we introduce remote repositories, you will learn about the concept of remote branches and remote-tracking branches.

To get a better idea of the concept of branches as movable pointers to commits, go to Follow Along 4-2.

[FOLLOW ALONG 4-2]

1. In a filesystem window, go to the rainbow project directory.
2. Reveal all the hidden files and directories in the rainbow project directory. For more information on how to view hidden files and directories, see "Viewing the Contents of Directories" on page 10.
3. Within the rainbow project directory in your filesystem window, go to .git > refs > heads > main.
4. Open the main file. On macOS, the file will automatically open in a basic text editor called TextEdit. On Microsoft Windows, you can choose to open the file with a basic text editor called Notepad.
   You will only use TextEdit and Notepad to view the contents of some of the files in the .git directory. These text editors are separate from the text editor you are using to manage the files in the Rainbow project.

What to notice:

- In step 4, inside the main file you will see the commit hash for the red commit in your rainbow repository.

In step 3 of Follow Along 4-2, in order to get to the main file, you navigated into the .git directory, the refs directory, and then the heads directory. The term "refs" stands for

"references." The heads directory stores a file for each local branch in your local repository. At the moment you only have one local branch, the main branch, so there is only one file in this directory. You can think of that file as storing the "head" of that branch; in other words, the latest commit on that branch.

Now that you understand a bit about what branches are and how they work, let's take a quick look at naming conventions for the primary branch.

A BIT OF GIT HISTORY: MASTER AND MAIN

Normally when you initialize a local repository using just the git init command with no options, behind the scenes Git creates a branch called master. However, "master" is not considered inclusive terminology, so in recent years, a large part of the Git community has decided to transition to using main (or other names) as the default branch name.

This is why in Chapter 2 when you initialized the rainbow repository, you used the git init command with the -b option and passed in the value main. The actual word *main* is not special in any way; you could have chosen to give the first branch any other name by passing in another value to the git init -b command. When you made the first commit in the rainbow repository, this updated the main branch to point to the first commit. In the Rainbow project, the main branch is the primary line of development.

In your Git learning journey you will come across many learning resources that still refer to the master branch. It is important to understand that there is nothing special about this branch; it's just the default name for the first branch created in Git.

At this point, you're ready to add a new color to the rainbowcolors.txt file and make another commit. But before we get to that, I'd like to introduce a few additional states that a file in a Git project can be in.

Unmodified and Modified Files

In Chapter 2, I introduced the concept of untracked files and tracked files. Git knows about the rainbowcolors.txt file because it has been included in a commit, and therefore it is a tracked file.

Tracked files in the working directory can be in one of two states. *Unmodified files* are files in the working directory that have not been edited since the last commit. Once a file in the working directory has been edited (and saved in the text editor), it becomes a

modified file. Since your last commit, you have not edited the rainbowcolors.txt file; therefore, it is an unmodified file.

[NOTE]

For Git to know that a file has been edited, the file *must* have been saved in the text editor. If you have made edits to a file but you have not saved those changes in your text editor, then Git will view it as an unmodified file.

In Chapter 3 you learned about the git status command, which shows you the state of the working directory and the staging area and the difference between the two. The git status command actually shows a list of all *modified* files and tells you whether or not they have been added to the staging area. It does not, however, list unmodified files.

Go to Follow Along 4-3 to use the git status command to see that the rainbowcolors.txt file is an unmodified file, then edit and save the file and use git status again to see how its state changes.

[FOLLOW ALONG 4-3]

1. rainbow $ **git status**
   On branch main
   nothing to commit, working tree clean

2. In the rainbow project directory in your text editor, open the rainbowcolors.txt file, add “Orange is the second color of the rainbow.” on line 2, and save the file.

3. rainbow $ **git status**
   On branch main
   Changes not staged for commit:
   (use "git add <file>..." to update what will be committed)
   (use "git restore <file>..." to discard changes in working directory)
   modified: rainbowcolors.txt
   no changes added to commit (use "git add" and/or "git commit -a")

What to notice:

- In step 1, the rainbowcolors.txt file is an unmodified file. It is not yet listed in the git status output.
- In step 3, the rainbowcolors.txt file is a modified file. It is now listed in the git status output.
- The rainbowcolors.txt file is not staged for commit; in other words, it has not been added to the staging area.

You just saw how the rainbowcolors.txt file went from being an unmodified file to a modified file when you edited it and saved your changes. See Figure 4-1 for an example of what your rainbowcolors.txt file should look like now.

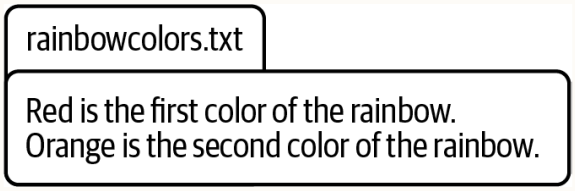


FIGURE 4-1

The rainbowcolors.txt file after you add a sentence about the color orange

In Follow Along 4-4, you will add the rainbowcolors.txt file to the staging area so that it can be included in your next commit, and you will observe how the git status output changes.

[FOLLOW ALONG 4-4]

1. rainbow $ **git add rainbowcolors.txt**

2. rainbow $ **git status**
   On branch main
   Changes to be committed:
   (use "git restore --staged <file>..." to unstage)
   modified: rainbowcolors.txt

What to notice:

- The rainbowcolors.txt file is staged for commit; in other words, it has been added to the staging area.

You have just completed the first step of the committing process, which is to add files to the staging area. Next, you'll make another commit and see how that affects your main branch.

Making Commits on a Branch

You are ready to make your second commit in the rainbow repository. This time you will add the color orange, so the commit message will be "orange". Go to Follow Along 4-5 to make your commit.

[NOTE]

If the output of the git log command extends beyond the size of the command line window, you must press Enter (Return) or use the down arrow to view the rest of the commits. To exit the command you must type Q.

[FOLLOW ALONG 4-5]

1. rainbow $ **git commit -m "orange"**
   [main 7acb333] orange
   1 file changed, 2 insertions(+), 1 deletion(-)

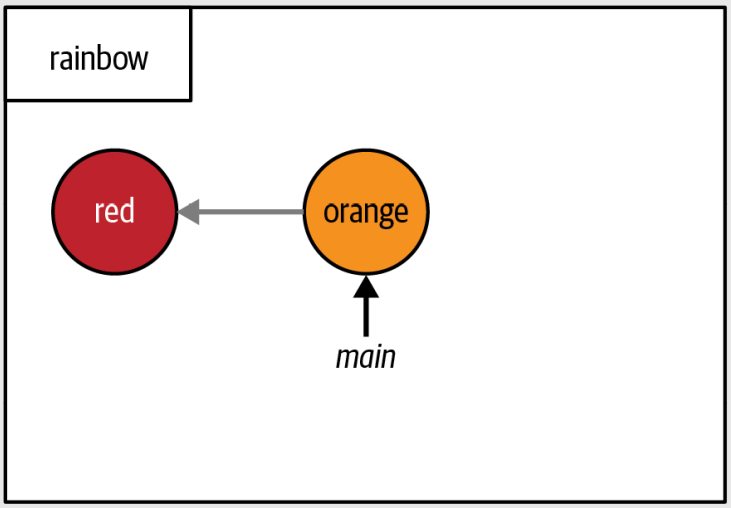
2. rainbow $ **git log**
   commit 7acb333f08e12020efb5c6b563b285040c9dba93 (**HEAD -> main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:42:07 2022 +0100
   orange
   commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:23:18 2022 +0100
   red

What to notice:

- You made a new commit, the orange commit. In the rainbow repository in this book the commit hash for the orange commit is 7acb333f08e12020efb5c6b563b285040c9dba93. Your commit hash will be different.
- The text HEAD -> main appears in parentheses next to the orange commit.

Visualize it 4-4 shows the state of the rainbow repository after Follow Along 4-5.

[VISUALIZE IT 4-4]



The rainbow repository after you make the orange commit

What to notice:

- There is a second commit, the orange commit.
- The orange commit points back to the red commit.
- The main branch points to the orange commit.

In Visualize it 4-4, you can see that there is a gray arrow pointing from the orange commit back to the red commit. This gray arrow represents the parent link. Every commit, other than the very first one in a repository, has a parent commit (some commits can have more than one parent; we will cover that in Chapter 5). The parent commit of the orange commit is the red commit; this is why the orange commit points back to the red commit.

These parent links describe how commits are linked to one another. Understanding these links allows you to visualize the commit history and keep track of what work has been done on which branches.

> **[NOTE]**
>
> In the Visualize It diagrams, gray arrows are used to represent parent links and black arrows are used to represent branch pointers.

To check which commit is the parent of a given commit, you can use the git cat-file command with the -p option and pass in a commit hash: git cat-file -p <commit_hash>. Most Git users probably won't use this command in their day-to-day work, but since it's a good learning tool, let's see it in action.

Go to Follow Along 4-6 to retrieve the commit hash of the parent of the orange commit. To do this, you'll need to pass in the orange commit's commit hash to the git cat-file -p command. The easiest way to retrieve the commit hash for a particular commit is to look at the list of commits produced by the git log command and copy the entire commit hash in the command line. Alternatively, you can look back at the git commit output in Follow Along 4-5 and pass in the first seven characters of the commit hash, which is shown there.

[FOLLOW ALONG 4-6]

1. Retrieve the commit hash for the orange commit (you can copy this from the git log output in Follow Along 4-5). You must pass this commit hash as an argument to the git cat-file -p command in step 2 of this Follow Along. You may copy and paste the entire commit hash or just enter the first seven characters, as shown here.

2. rainbow $ **git cat-file -p 7acb333**
   tree 407fe6a858cd7f157405e013a088fdc1c61f0a40
   parent c26d0bc371c3634ab49543686b3c8f10e9da63c5
   author annaskoulikari <gitlearningjourney@gmail.com> 1645260127 +0100
   committer annaskoulikari <gitlearningjourney@gmail.com> 1645260127 +0100
   orange

What to notice:

- In the git cat-file -p output, you can see that next to parent it references the commit hash of the red commit in this book. In your output, it will reference the commit hash of your red commit.

You've now made a second commit in your rainbow repository, and in Visualize it 4-4 you observed that as you make commits on a branch, the branch pointer moves to point to the latest commit. Next, let's go over how to make other branches so that you can work on other lines of development.

Creating a Branch

At the moment, you only have one local branch, called main, in your rainbow repository. To list the branches in a local repository, you can use the git branch command. To create a new branch, you can pass the name of a branch that doesn't exist yet to this command. Note that branch names cannot contain spaces.

[SAVE THE COMMAND]

git branch

List local branches

git branch <new_branch_name>

Create a branch

As with commit messages, as discussed in Chapter 3, if you're working with other people on a project you should check if there are any existing rules that determine how to name your branches. Since there are no specific branch naming rules in the Rainbow project, you will use the generic name feature for the branch you will create in Follow Along 4-7. Keep in mind, however, that in a real Git project the branch name would usually be more descriptive, relating to the feature or topic you're working on.

[FOLLOW ALONG 4-7]

1. rainbow $ **git branch**
   * main

2. rainbow $ **git branch feature**

3. rainbow $ **git branch**
   feature
   * main

4. rainbow $ **git log**
   commit 7acb333f08e12020efb5c6b563b285040c9dba93 (**HEAD -> main**, **feature**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:42:07 2022 +0100
   orange
   commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:23:18 2022 +0100
   red

5. Using your filesystem window, go to .git > refs > heads to see which files are in there now.

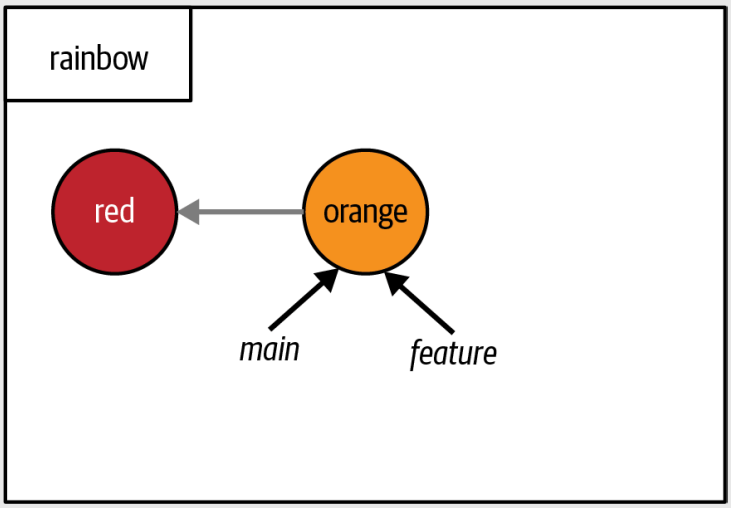
6. Open the feature file. The file should contain a commit hash.

What to notice:

- In step 2, you made a new branch called feature that points to the orange commit.

The state of the rainbow repository after you complete Follow Along 4-7 is illustrated in Visualize it 4-5.

[VISUALIZE IT 4-5]



The rainbow repository after you make the feature branch

In Visualize it 4-5 you can see that there are now two arrows, representing the main and feature branches, pointing to the orange commit. A new branch will initially point to the commit that you were on when you made the branch. In this case, you can say that you "made the feature branch off of the main branch." That is why the feature branch and main branch both now point to the same commit.

In the git log output from Follow Along 4-7, in the parentheses next to the orange commit, you should see HEAD —> main, feature. As you know, main and feature are branches —but what is HEAD?

What Is HEAD?

At any given point in time, you are looking at a particular version of your project. Therefore, you are on a particular branch which is pointing to a commit. HEAD is simply a pointer that tells you which branch you are on. The name HEAD is always in capital letters, but this is simply a convention; it is not an acronym.

[NOTE]

There are times when you can be on a commit that is not pointed to by a branch. Git calls this “detached HEAD state.” We’ll explore this further in “Checking Out Commits” on page 80.

Go to Follow Along 4-8 to explore what HEAD is by taking a look at the .git directory.

[FOLLOW ALONG 4-8]

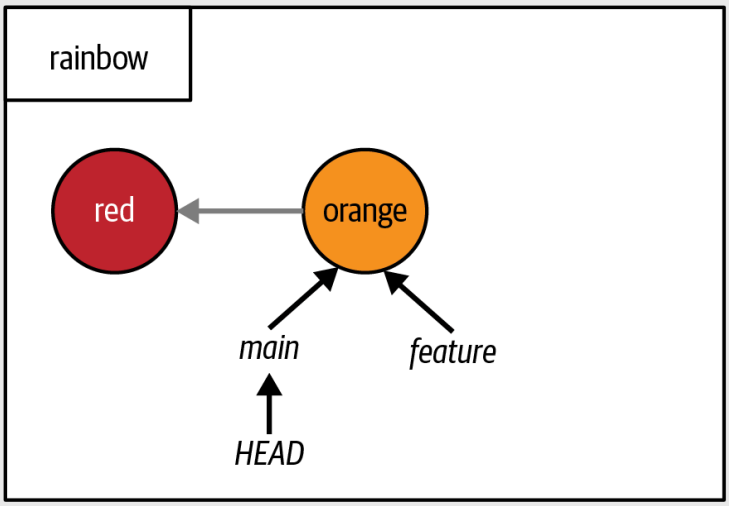
1. Using your filesystem window, go to rainbow > .git > HEAD.
2. Open the HEAD file. The contents should be ref: refs/heads/main.

What to notice:

- The HEAD file contains ref: refs/heads/main, which is a reference to the main file that represents the main branch.

This is illustrated in Visualize it 4-6.

[VISUALIZE IT 4-6]



In the rainbow repository, HEAD is pointing to main

In Visualize it 4-6, an arrow shows HEAD is pointing to main. This indicates you're currently on the main branch.

[NOTE]

HEAD (in capital letters) should not be confused with the heads directory that can be found in .git > refs > heads. The heads directory stores a file for every local branch in your local repository, while HEAD indicates which branch you are on by referencing one of the files inside the heads directory. You can distinguish them because HEAD is always in capital letters.

Another way of knowing which branch you're currently on is to look at the output of either the git branch command or the git log command. In the git branch output, the branch you are currently on will have an asterisk next to it. If you look at the output in Follow Along 4-7, you can see the asterisk is next to the main branch. In the git log output, HEAD will point to the branch you are on inside the parentheses.

Now that you've created a new branch, you're ready to start using it—but for the time being, you're still on the main branch. Next, you will switch branches, moving the HEAD pointer to your new feature branch.

Switching Branches

To work on another branch (or line of development) in a Git project, you have to switch onto that branch. Another way of saying this in Git terminology is that you have to "check out" another branch.

You currently have two branches, main and feature. But as you've just seen, just because you make a branch in Git does not mean that you automatically switch onto that branch. You must explicitly instruct Git that you want to switch onto a branch. You can do this using either the git switch command or the git checkout command, passing in the name of the branch that you want to switch onto.

[NOTE]

If you have a version of Git that is older than version 2.23, then you won't have access to the git switch command and you must use the git checkout command. The git checkout command is available to all Git users.

[SAVE THE COMMAND]

git switch *<branch_name>*

Switch branches

git checkout *<branch_name>*

Switch branches

The only purpose of the git switch command is to switch branches, while the git checkout command can do more things. We will talk more about git checkout in "Checking Out Commits" on page 80.

From now on, in the Follow Alongs in this book you will use the git switch command, as this is the specialized command included in the latest versions of Git for this purpose. However, you can always choose to use the git checkout command instead, as they are equivalent.

The git switch (or git checkout) command does three things when used to switch branches:

1. It changes the HEAD pointer to point to the branch you are switching onto.
2. It populates the staging area with a snapshot of the commit you are switching onto.
3. It copies the contents of the staging area into the working directory.

> **[NOTE]**
>
> If you need a refresher on what the staging area and working directory are, refer back to "The Areas of Git" on page 26.

In short, when you change branches you end up changing the commit that you're looking at, provided that the two branches point to two different commits. At the moment, both of the branches you have in the rainbow repository point to the same commit, so only the first action will take place, and the commit you are on will not change. In Chapter 5, you will go over an example where all three of the actions listed here take place.

Go to Follow Along 4-9 to switch onto the feature branch.

[FOLLOW ALONG 4-9]

1. rainbow $ **git branch**
   feature
   * main

2. rainbow $ **git switch feature**
   Switched to branch 'feature'

3. rainbow $ **git branch**
   * feature
   main

4. rainbow $ **git log**
   commit 7acb333f08e12020efb5c6b563b285040c9dba93 (**HEAD -> feature**, **main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:42:07 2022 +0100
   orange
   commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:23:18 2022 +0100
   red

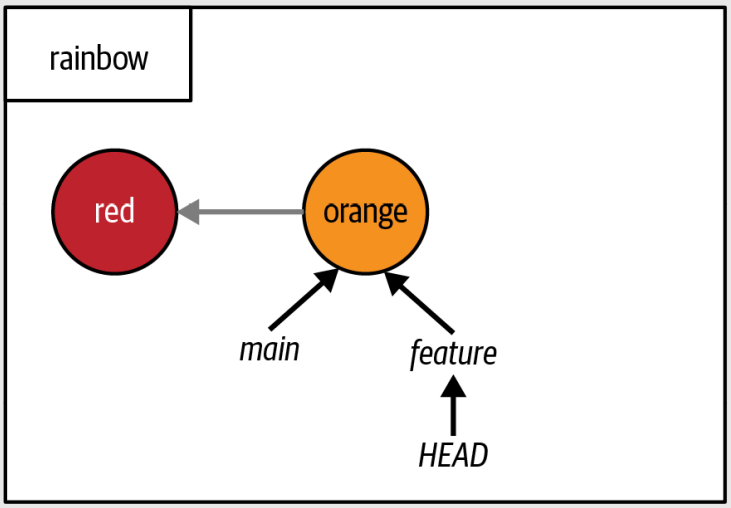
5. Using your filesystem window, go to rainbow > .git > HEAD, and open the HEAD file in a new window. You will see that the contents of the file have changed and it now refers to the feature branch, refs/heads/feature.

What to notice:

- In step 4, the git log output shows that HEAD points to the feature branch.
- You have switched onto the feature branch, so this is where you'll be working now.

These observations are illustrated in Visualize it 4-7.

[VISUALIZE IT 4-7]



The rainbow repository after you switch onto the feature branch

As you can see, switching branches changed the contents of the HEAD file in the .git directory. This is illustrated in Figure 4-2.

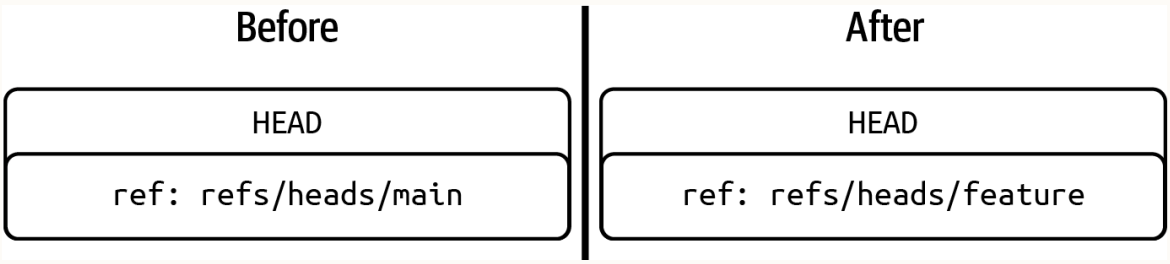


FIGURE 4-2

The contents of the HEAD file before and after you switch from the main branch onto the feature branch in the rainbow repository

Now that you have switched onto the feature branch, let's see what happens when you work on it.

Working on a Separate Branch

You are now on the feature branch, and in Follow Along 4-10 you're going to add the color yellow to the rainbow repository.

[FOLLOW ALONG 4-10]

1. In the rainbow project directory in your text editor, in the rainbowcolors.txt file, add "Yellow is the third color of the rainbow." on line 3. Then save the file.

2. rainbow $ **git add rainbowcolors.txt**

3. rainbow $ **git commit -m "yellow"**
   [feature fc8139c] yellow
   1 file changed, 2 insertions(+), 1 deletion(-)

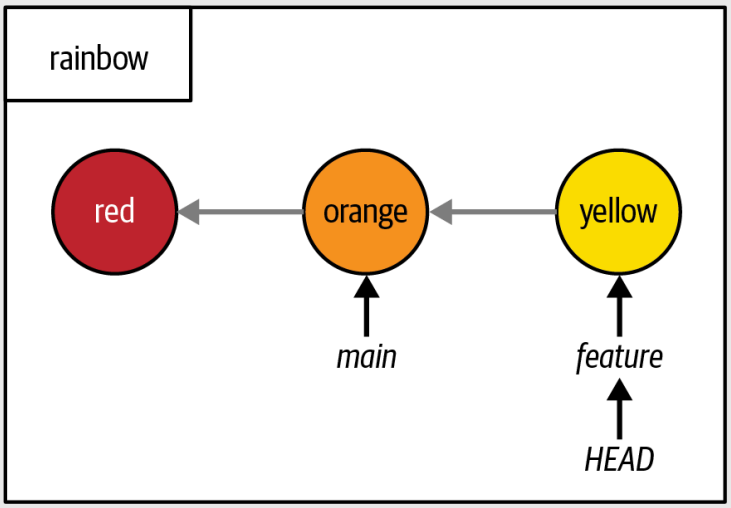
4. rainbow $ **git log**
   commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> feature**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 10:09:59 2022 +0100
   yellow
   commit 7acb333f08e12020efb5c6b563b285040c9dba93 (**main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:42:07 2022 +0100
   orange
   commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:23:18 2022 +0100
   red

What to notice:

- The feature branch points to the latest commit, the yellow commit.
- The main branch still points to the orange commit.

These observations are illustrated in Visualize it 4-8.

[VISUALIZE IT 4-8]



The rainbow repository after you make the yellow commit

As mentioned previously, when you make a commit, it is the branch you're currently on that updates to point to the new commit. The main and feature branches no longer point to the same commit because feature has updated to point to the new yellow commit. HEAD continues to point to the feature branch.

Summary

This chapter introduced the concept of branches as different lines of development and showed how, in practice, they are movable pointers to commits. We explored why you might want to use them, and you learned how to list, create, and change branches. We covered why the first branch in the rainbow repository is called main, and why in other learning resources teaching Git you may come across a branch called master. You also learned that HEAD is a pointer to the branch you're currently on.

While making more commits, you observed how tracked files change state from unmodified to modified when they are edited. You also observed that it's the branch you're on that moves to point to the latest commit you make in a local repository, and you saw how commits are linked to each other through parent links.

Now that we've covered how to use branches to work on different lines of development simultaneously, let's continue on to Chapter 5 to start learning how to combine these different lines of development by merging.

Previous chapter

< Chapter 3. Making a Commit

Next chapter

Chapter 5. Merging >